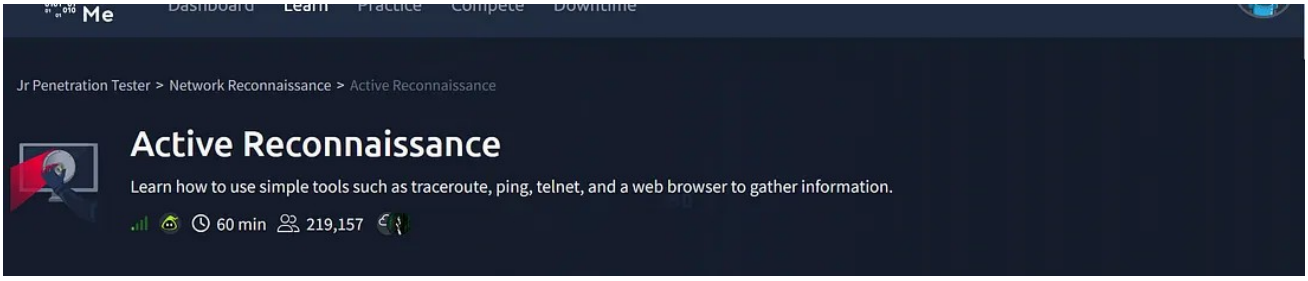
Open in app ↗

Medium

Access to everything. Now 30% off. Upgrade now



Active Reconnaissance +

Active Reconnaissance " TRYHACKME WALKTROUGH"

20 min read · Just now

Nuh hozar

Listen Share More

Task 1- Introduction

Active reconnaissance is the process of directly interacting with a target system or network to gather information about it. Passive reconnaissance collects data from public sources without sending any traffic to the target. Active reconnaissance, by contrast, requires transmitting packets, making connections, and probing services. The distinction matters because active techniques leave traces in the form of log entries, IDS alerts, WAF blocks, and honeypot triggers.

The previous room on passive reconnaissance covered gathering intelligence from DNS records, WHOIS data, certificate logs, and services like Shodan without the target ever knowing. This room shifts to direct engagement. You will use web browsers, `ping`, `traceroute`, `telnet`, and `netcat` to interact with targets and extract information that passive methods cannot reveal.



Passive reconnaissance involves checking public data such as DNS records, WHOIS results, certificate logs, and Shodan queries without sending packets to the target.



Active reconnaissance involves making direct contact by visiting websites, pinging hosts, tracing routes, and connecting to ports. This can reveal live services, open ports, banners, and network paths, but it leaves footprints.

Prerequisites: This room assumes familiarity with core networking concepts such as TCP, UDP, ICMP, and port numbers. If these terms are unfamiliar, complete the Networking module before proceeding.

Active reconnaissance is riskier and more detectable than it has ever been. Most organisations deploy CDNs such as Cloudflare and Akamai alongside WAFs and zero-trust models that log or outright block unusual probes. IPv6 adoption means many hosts respond to `ping6` but filter ICMPv4. HTTPS dominates web traffic, making plaintext protocols largely obsolete for web interactions. On the defensive side, SIEMs, EDR solutions, and cloud-native monitoring detect reconnaissance patterns quickly and reliably.

Despite the dominance of encrypted protocols, tools like `telnet` remain worth understanding. They illustrate the fundamentals of banner grabbing and cleartext protocol interaction, which helps you appreciate why modern alternatives such as `netcat` and `curl` are preferred. You will also encounter telnet-based weaknesses in legacy environments, so knowing how it works has practical value.

Critical rule: Never perform active reconnaissance without explicit, signed legal authorisation such as a penetration testing contract or bug bounty scope. Unauthorised probing is illegal in most jurisdictions.

From a red team perspective, the goal is to blend in. A web browser visiting a website looks identical to normal traffic among thousands of legitimate users, and crafted requests with realistic User-Agent strings and slow timing can sometimes evade basic detection. From a blue team perspective, active probes surface in access logs, firewall logs, WAF events, and IDS alerts. Monitoring your own exposure helps catch reconnaissance early.

Learning Objectives

By the end of this room, you will be able to:

- Use a web browser and its Developer Tools to inspect headers, JavaScript source files, and certificates for reconnaissance purposes
- Use `ping` to test host reachability and infer operating system details from TTL values
- Use `traceroute` and `mtr` to map network paths and discover intermediate hops
- Use `telnet` for legacy banner grabbing and understand why modern alternatives are preferred
- Use `netcat` (`nc`) to perform banner grabbing, basic port probing, and simple client-server communication

These tools are straightforward and often pre-installed on Linux distributions such as the AttackBox, Kali, and Parrot. This room is suitable for beginners

looking to understand active reconnaissance fundamentals before progressing to the Nmap rooms.

Task 2- Web Browser

The web browser is one of the most convenient and least suspicious tools for active reconnaissance. It is present on virtually every system, and its traffic blends in with normal user activity. This makes it difficult for defenders to distinguish reconnaissance from legitimate browsing.

Transport-Level Basics

Browsers connect to **TCP port 80** by default for plain HTTP, though this is now rare because almost all sites automatically redirect to HTTPS. **TCP port 443** is the standard for HTTPS and is used by nearly all websites today.

Many modern sites also support **HTTP/3**, which uses the **QUIC** protocol. QUIC is a transport protocol originally developed by Google that combines the functions of TCP and TLS into a single protocol running over **UDP port 443**. The result is faster and more reliable connections than traditional TCP+TLS. You can identify HTTP/3 traffic in the browser's Network tab, where the protocol column displays `h3`.

You can access services on non-standard ports by specifying them explicitly in the URL. For example, navigating to `https://target.com:8443/` or `http://192.168.1.100:8080/` will cause the browser to attempt a connection on that port. If a web server is listening, the page will load.

Developer Tools

Press **Ctrl + Shift + I** on Windows and Linux, or **Option + Command + I** on macOS, in Firefox, Chrome, Edge, or most Chromium-based browsers to open Developer Tools. Several of the tabs available are directly useful for reconnaissance.

The **Network** tab displays all requests and responses in real time. This includes request and response headers such as `Server`, `X-Powered-By`, and `Content-Security-Policy`, along with timing data, status codes, and cookies sent and received.

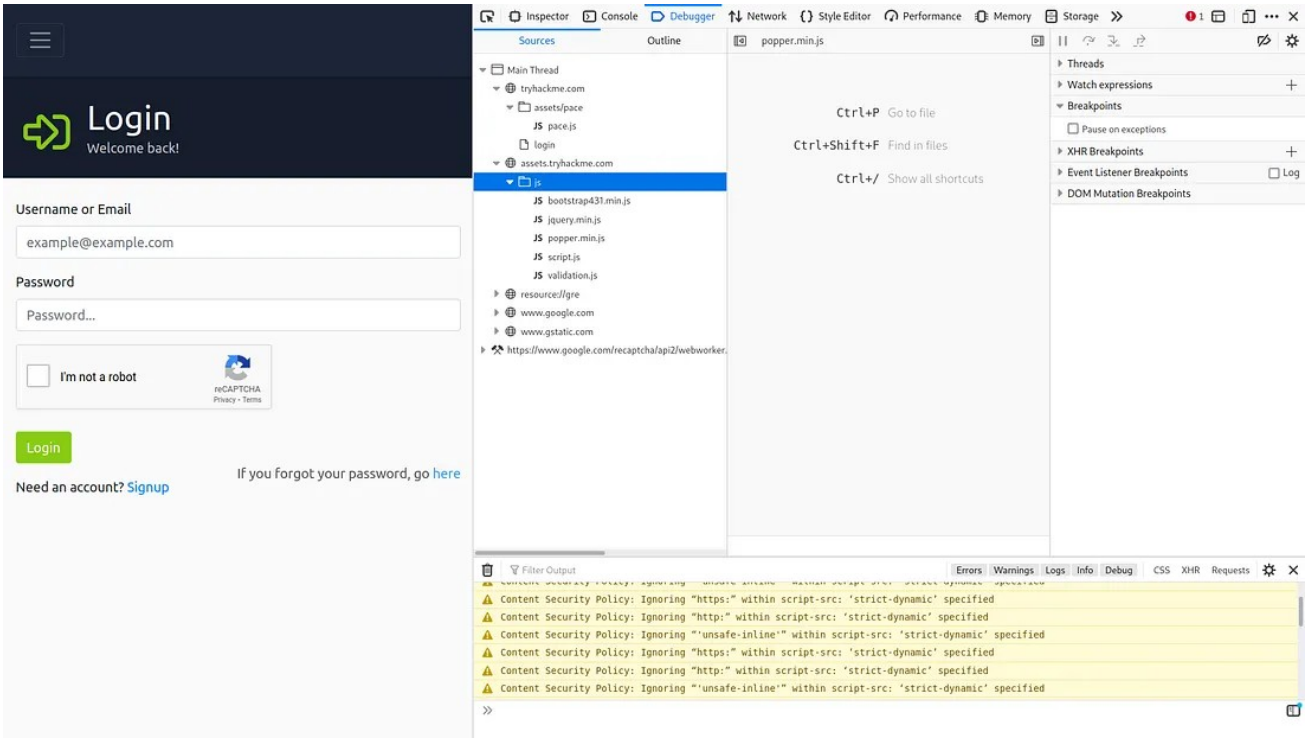
The **Console** tab allows you to execute JavaScript snippets directly in the page context, view errors, and interact with the DOM.

The **Sources** tab lets you browse JavaScript, CSS, and HTML files loaded by the page. This is one of the most practical reconnaissance techniques available through the browser. JavaScript source files frequently contain hardcoded API endpoints, directory structures, references to internal services, and developer comments that were never intended to be public. Inspecting these files can surface information that is invisible in the rendered page. You will use this technique in the questions below.

The **Application** tab, under the Storage section, lets you inspect cookies, Local Storage, and Session Storage. These storage areas sometimes contain session tokens, API keys accidentally exposed client-side, tracking IDs, or authentication data.

The **Security** tab provides certificate details including the issuer, validity period, and Subject Alternative Names (SANs). SANs frequently reveal additional subdomains or related domains belonging to the same organisation.

Below is a screenshot of Firefox Developer Tools. Chrome and Edge provide a very similar interface.



Browser Extensions

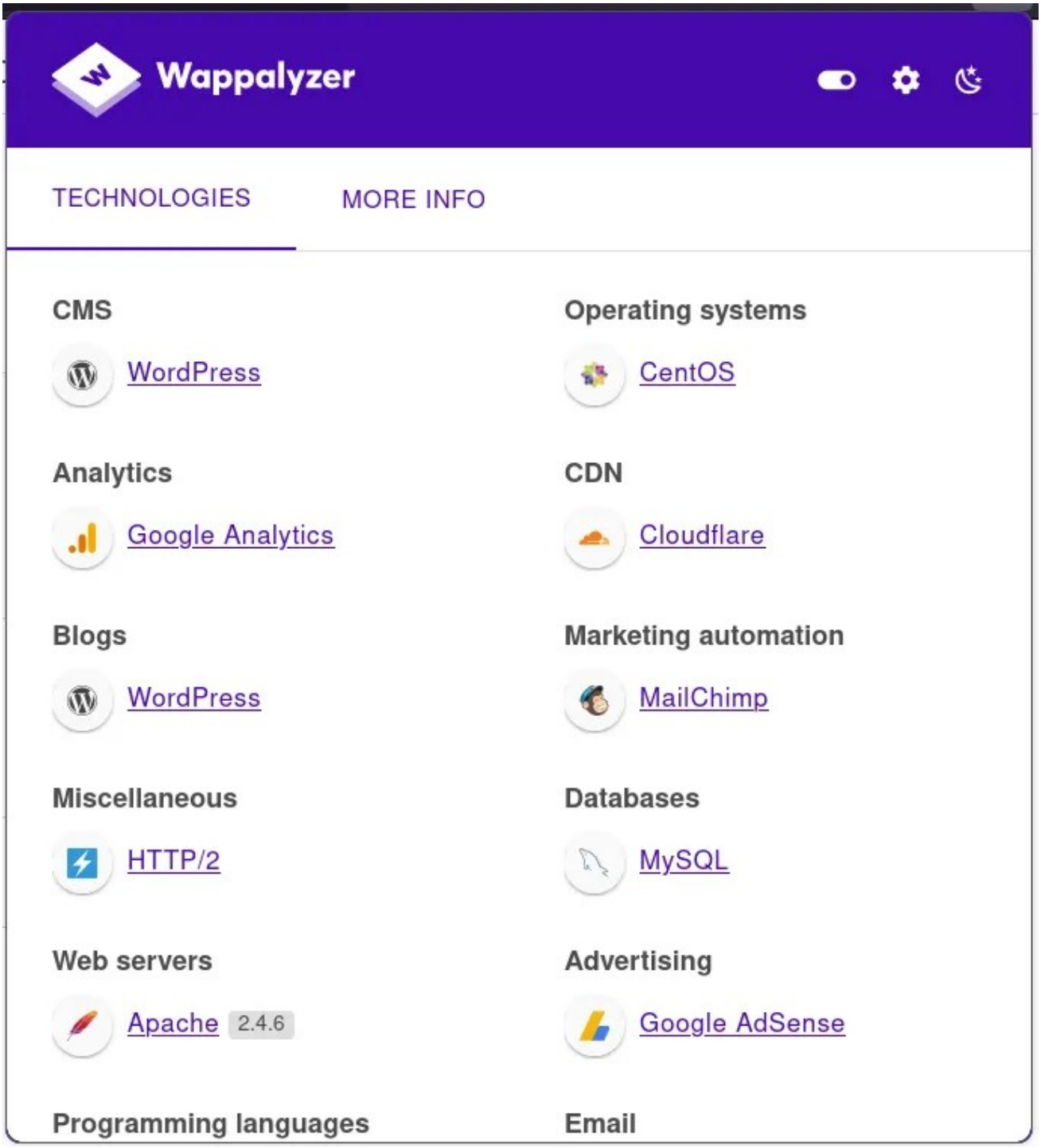
Browser extensions can extend the browser into a more capable reconnaissance

platform. The following are currently popular and actively maintained.

FoxyProxy allows you to switch between proxies such as Burp Suite, ZAP, and SOCKS5 tunnels. This is useful when intercepting or routing traffic through different tools during an engagement.

User-Agent Switcher and Manager changes the User-Agent string to emulate different browsers, operating systems, or devices. You can present as mobile Safari or an older browser version to discover mobile-specific endpoints or version-specific behaviour. However, many modern WAFs and CDNs detect suspicious or rapid User-Agent changes.

Wappalyzer automatically identifies technologies used on the site, including CMS platforms, web servers, JavaScript frameworks, analytics tools, CDNs, and databases. It runs passively while you browse and is one of the most widely used extensions for quick technology fingerprinting.



Other useful alternatives include **BuiltWith Technology Profiler**, which is similar to Wappalyzer but sometimes detects different technologies, **WhatRuns** as a lightweight alternative, and **Library Detector for Chrome/Firefox**, which focuses specifically on JavaScript libraries and frameworks. Over time, most people settle on a small set of three to five extensions that best match their workflow.

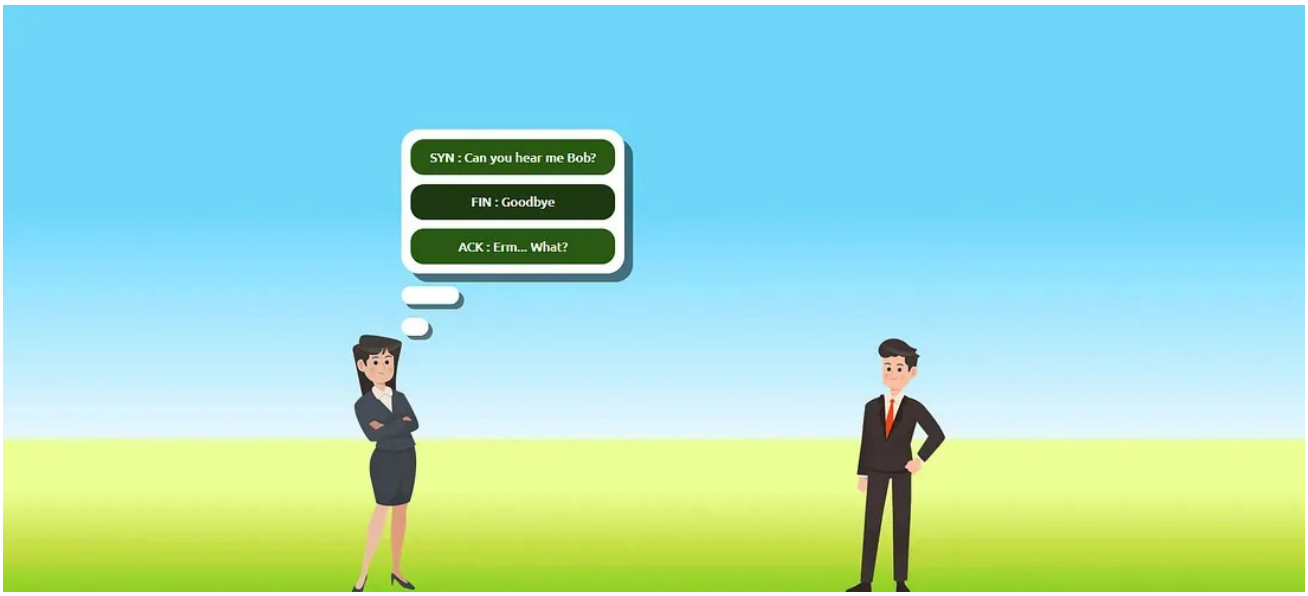
Even though browsing appears normal, unusual patterns can still trigger behavioural rules in modern WAFs or endpoint detection systems. Rapid page

loads, modified headers, frequent DevTools usage, and abnormal User-Agent strings are all signals that defenders look for. The goal is to mimic legitimate user behaviour whenever possible.

Answer the questions below

Q 1-Browse to the following website and ensure that you have opened your Developer Tools on AttackBox Firefox, or the browser on your computer. Using the Developer Tools, figure out the total number of questions.

Answer : 8



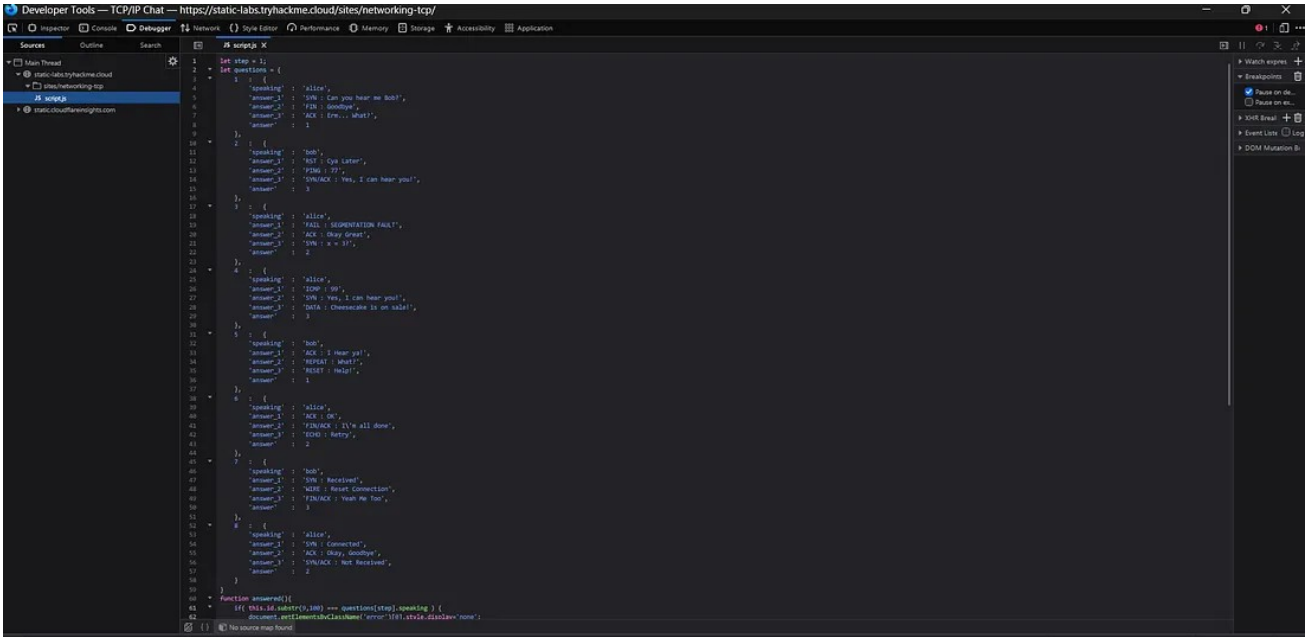
steps for a solution

1- at first click the hyperlinked

2- The website involves Javascript so I pressed **CTRL + SHIFT + I** then the Developer Tools appears on the right side and click Debugger at firefox developer tools

3-Go to Debugger and you'll see a hierarchy files you can easily locate the **script.js** (this tells on the hint)

4-The **script.js** has some following function codes.



Task 3- Ping

The name **ping** comes from the sound of a sonar pulse. You send out a signal and listen for the echo to come back. In networking, the `ping` command does the same thing. It sends a small test packet to a remote host and waits for a reply. This simple exchange tells you whether the target is reachable over the network and whether it is online and responding.

How Ping Works

Ping uses the **ICMP protocol** (Internet Control Message Protocol). It sends an **ICMP Echo Request** packet (type 8). If the target receives the packet and is permitted to answer, it sends back an **ICMP Echo Reply** (type 0). This exchange is very lightweight and fast, which is why ping became the standard first check before spending time on more detailed scanning.

Basic Usage

On Linux and macOS, use the `-c` flag to specify the number of packets to send.

```
ping -c 5 10.113.178.176
```

You can also ping a hostname, in which case DNS resolution happens first.

```
ping -c 5 tryhackme.com
```

On Windows, the equivalent flag is `-n`.

```
ping -n 5 10.113.178.176
```

If you omit the count on Linux, ping runs indefinitely. Press **Ctrl+C** to stop it.

You can force a specific IP version using the `-4` and `-6` flags. This is useful in dual-stack environments where a hostname resolves to both IPv4 and IPv6 addresses. On some systems, `ping6` is available as a standalone command for IPv6.

```
ping -4 -c 5 10.113.178.176
ping -6 -c 5 MACHINE_IPV6
```

Interpreting the Output: Successful Ping

The following example shows a target that is alive and allows ICMP.

```
user@AttackBox$ ping -c 5 10.113.178.176
PING 10.113.178.176 (10.113.178.176) 56(84) bytes of data.
64 bytes from 10.113.178.176: icmp_seq=1 ttl=64 time=0.512 ms
64 bytes from 10.113.178.176: icmp_seq=2 ttl=64 time=0.478 ms
64 bytes from 10.113.178.176: icmp_seq=3 ttl=64 time=0.491 ms
64 bytes from 10.113.178.176: icmp_seq=4 ttl=64 time=0.503 ms
64 bytes from 10.113.178.176: icmp_seq=5 ttl=64 time=0.485 ms
```

```
--- 10.113.178.176 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4098ms
rtt min/avg/max/mdev = 0.478/0.494/0.512/0.012 ms
```

The target answered all five requests, confirming it is online and reachable. The 0% packet loss confirms a clean network path. The round-trip time of approximately 0.5 ms is very low, indicating the target is likely on the same local network.

The **TTL** (Time To Live) field deserves particular attention. Although "Time" appears in the name, TTL actually represents the maximum number of routers (hops) a packet can pass through before being dropped. Each router along the path decrements the TTL by one. The initial TTL value is set by the operating system, which makes it a useful indicator for OS fingerprinting. Linux typically uses a starting TTL of 64, while Windows typically uses 128.

However, intermediate routers decrement this value before it reaches you. A TTL of 58 in a response likely indicates a Linux system that is six hops away, not a different operating system entirely. Keep this in mind when interpreting results.

Interpreting the Output: No Reply

```
user@AttackBox$ ping -c 5 10.113.178.176
PING 10.113.178.176 (10.113.178.176) 56(84) bytes of data.
From ATTACKBOX_IP icmp_seq=1 Destination Host Unreachable
From ATTACKBOX_IP icmp_seq=2 Destination Host Unreachable
From ATTACKBOX_IP icmp_seq=3 Destination Host Unreachable
From ATTACKBOX_IP icmp_seq=4 Destination Host Unreachable
From ATTACKBOX_IP icmp_seq=5 Destination Host Unreachable
```

```
--- 10.113.178.176 ping statistics ---
5 packets transmitted, 0 received, +5 errors, 100% packet loss,
time 4098ms
```

There are several common reasons for receiving no reply. The lab machine may be powered off, crashed, or still booting. A router or firewall along the path may

be blocking ICMP Echo Requests. The target may be behind NAT that drops ICMP. Windows Firewall blocks ping by default on most versions. Corporate firewalls, cloud providers such as AWS, Azure, and GCP, and modern WAFs and CDNs frequently block ICMP completely. Your own network or machine may also be blocking outgoing ICMP.

Quick Reference

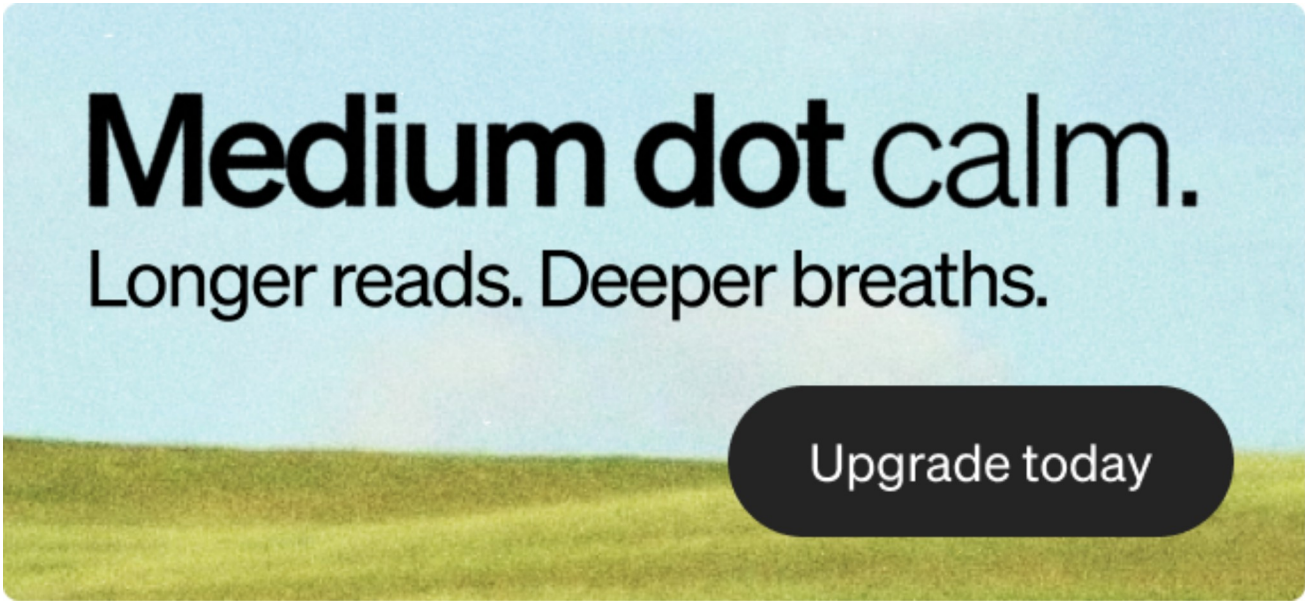
Result	Most likely meaning	Next step
Fast replies, low or no packet loss	Target is online and allows ICMP	Proceed to port scanning
"Destination Host Unreachable"	Target is down or no route exists	Check if machine is powered on
100% packet loss with no error message	ICMP is filtered or blocked	Try TCP/UDP host discovery with Nmap
High latency or heavy	Network congestion, long distance, or filtering	Investigate the path with `traceroute`



Q 2-Which option would you use to set the size of the data carried by the ICMP echo request?

Answer : -s

Linux and macos we can -s but windows wecan -l



Q 3-What is the size of the ICMP header in bytes?

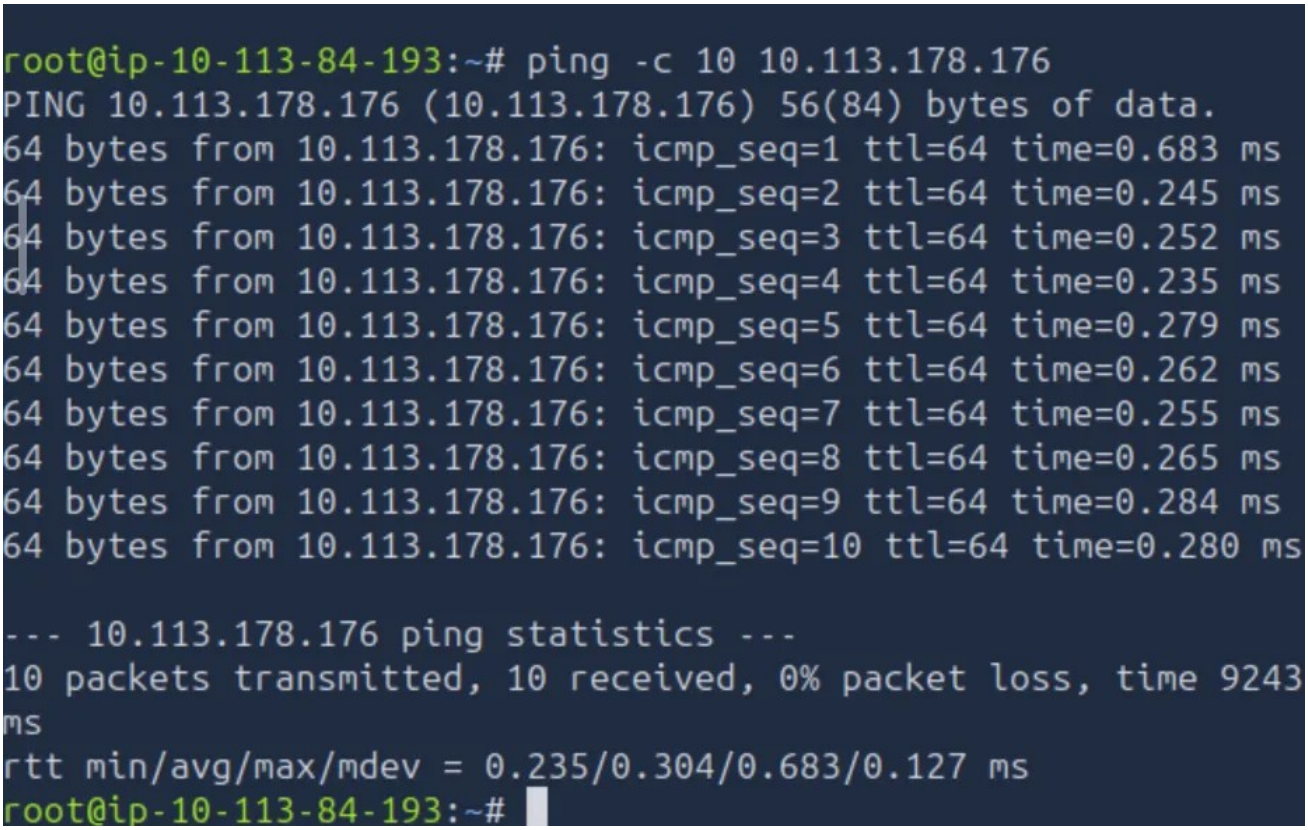
Answer : 8

Q 4- Does MS Windows Firewall block ping by default? (Y/N)

Answer : Y

Q 5- Deploy the VM for this task and using the AttackBox terminal, issue the command `ping -c 10 10.113.178.176` . How many ping replies did you get back?

Answer : 10

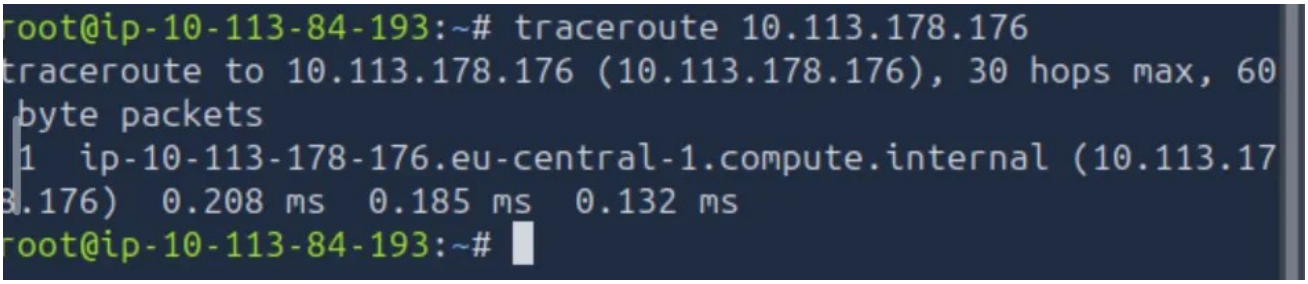


Task 4- Tracerout

The `traceroute` command traces the route that packets take from your system to a target host. Its purpose is to discover the IP addresses of the routers (hops) along the path and to determine how many of them sit between you and the destination. This information is useful for understanding network topology, identifying where filtering or latency occurs, and mapping infrastructure.

However, the route packets take is not fixed. Many routers use dynamic routing protocols such as BGP or OSPF that adapt to network changes. Modern networks also employ load balancing and anycast routing, meaning packets can take different paths even in consecutive runs of the same command.

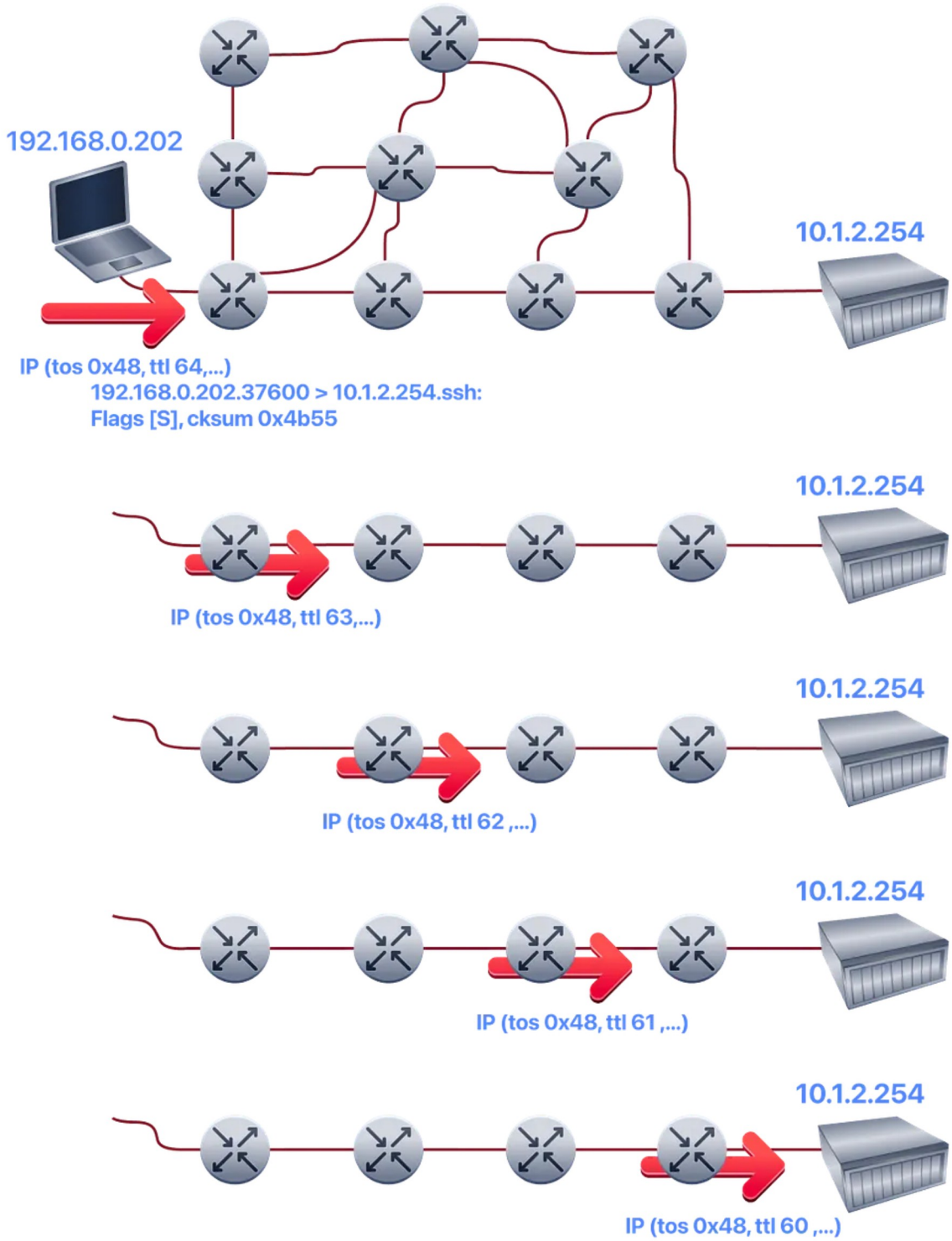
On Linux and macOS, the command is `traceroute 10.113.178.176`. On Windows, it is `tracert 10.113.178.176`. For IPv6, use `traceroute -6 MACHINE_IPV6` or the standalone `traceroute6` command.



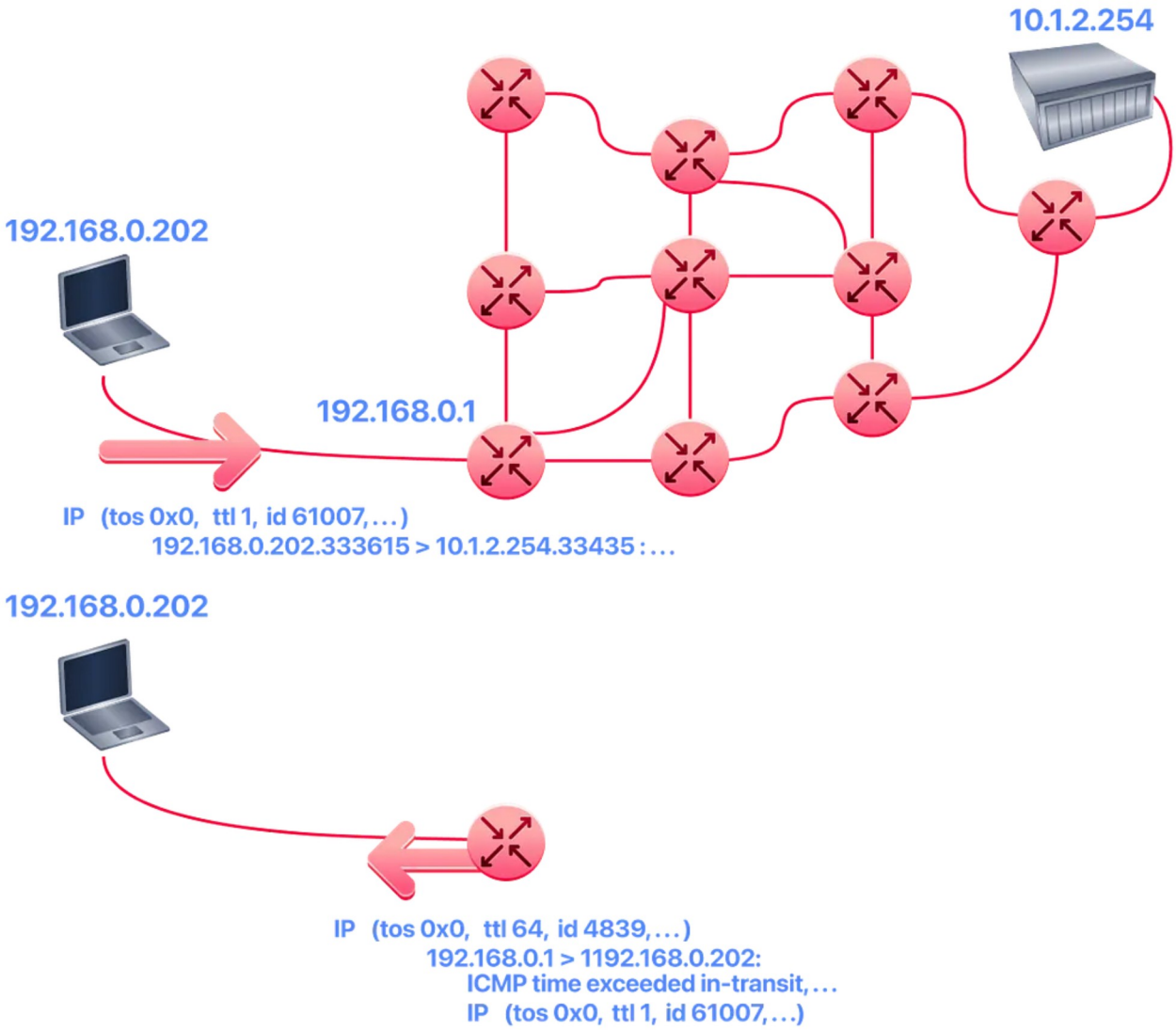
How Traceroute Works

There is no direct way to discover the full path from your system to a target. Instead, `traceroute` exploits the **TTL** (Time To Live) field in the IP header. Each router that handles a packet decrements the TTL by one before forwarding it. When the TTL reaches 0, the router drops the packet and sends an **ICMP Time-to-Live Exceeded** message back to the sender.

The following diagram illustrates a packet leaving a system with a TTL of 64. After passing through four routers, each decrementing the TTL by one, the packet arrives at the target with a TTL of 60.

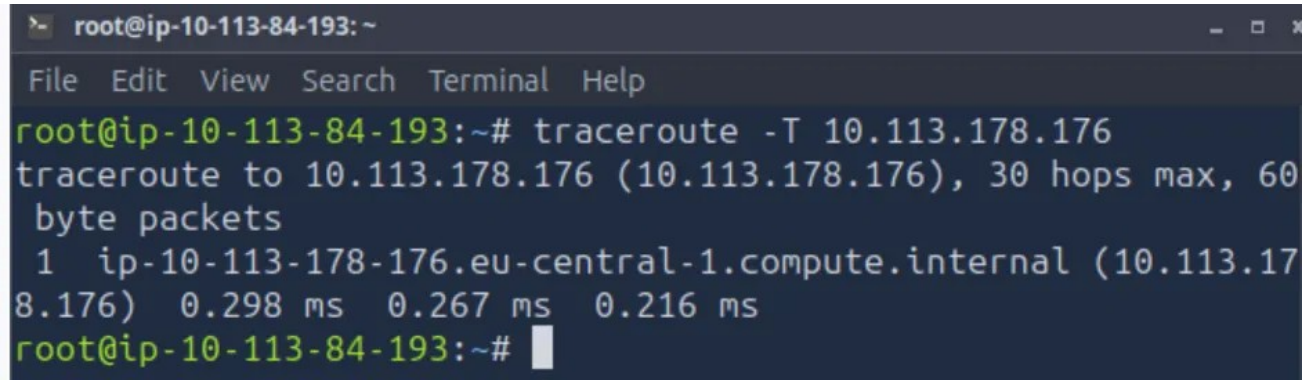


By sending packets with incrementally increasing TTL values starting at 1, `traceroute` forces each successive router along the path to reveal its IP address. A TTL of 1 causes the first router to drop the packet and respond. A TTL of 2 reaches the second router before being dropped. This continues until the packet reaches the destination.

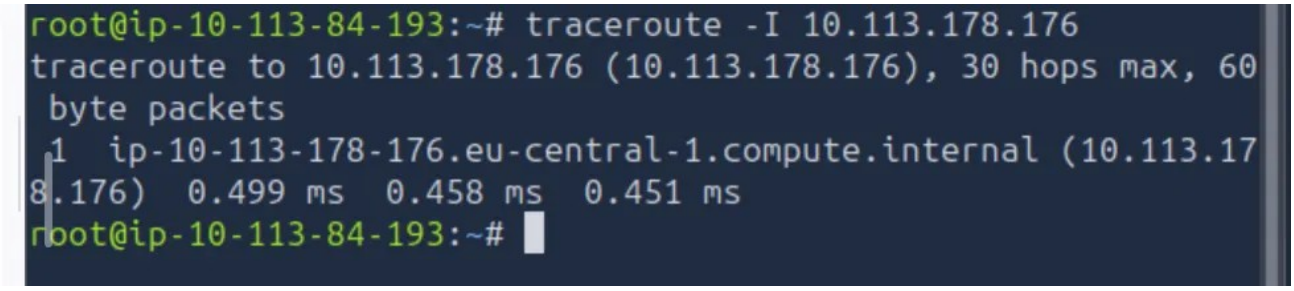


Some routers are configured not to send ICMP Time-to-Live Exceeded messages. This is common in secure environments intended to prevent reconnaissance. These routers appear as `*` in the traceroute output.

On Linux, `traceroute` sends UDP datagrams by default. To switch to TCP-based tracing, which is useful for bypassing UDP filters, use `traceroute -T 10.113.178.176`. For ICMP-based tracing, use `traceroute -I 10.113.178.176`.



traceroute -T



traceroute -I

Reading the Output

The following two examples show `traceroute tryhackme.com` run twice from TryHackMe's AttackBox. They demonstrate how routes can change between consecutive runs.

Traceroute A

AttackBox Terminal — Traceroute A

```
user@AttackBox$ traceroute tryhackme.com
traceroute to tryhackme.com (172.67.69.208), 30 hops max, 60 byte packets
 1 ec2-3-248-240-5.eu-west-1.compute.amazonaws.com (3.248.240.5) 2.663 ms * (
 2 100.66.8.86 (100.66.8.86) 43.231 ms 100.65.21.64 (100.65.21.64) 18.886 ms
 3 * 100.66.16.176 (100.66.16.176) 8.006 ms *
 4 100.66.11.34 (100.66.11.34) 17.401 ms 100.66.10.14 (100.66.10.14) 23.614 r
 5 100.66.7.35 (100.66.7.35) 12.808 ms 100.66.6.109 (100.66.6.109) 14.791 ms
 6 100.65.14.131 (100.65.14.131) 1.026 ms 100.66.5.189 (100.66.5.189) 19.246
 7 100.65.13.143 (100.65.13.143) 14.254 ms 100.95.18.131 (100.95.18.131) 0.94
 8 100.95.2.143 (100.95.2.143) 0.680 ms 100.100.4.46 (100.100.4.46) 1.392 ms
 9 100.100.20.76 (100.100.20.76) 7.819 ms 100.92.11.36 (100.92.11.36) 18.669
10 100.92.11.112 (100.92.11.112) 17.852 ms * 100.92.11.158 (100.92.11.158) 1(
11 100.92.211.82 (100.92.211.82) 19.713 ms 100.92.0.126 (100.92.0.126) 18.60
```

```
12 99.83.69.207 (99.83.69.207) 17.603 ms 15.827 ms 17.351 ms
13 100.92.9.83 (100.92.9.83) 17.894 ms 100.92.79.136 (100.92.79.136) 21.250 m
14 172.67.69.208 (172.67.69.208) 17.976 ms 16.945 ms 100.92.9.3 (100.92.9.3)
```

This output contains 14 numbered lines, each representing one hop. The system sends three packets at each TTL value, which is why you see up to three IP addresses and three round-trip times per line. Some lines show different IP addresses for the same hop number because load balancing caused each of the three packets to take a slightly different path.

Consider line 12. The router at `99.83.69.207` responded to all three packets, with round-trip times of 17.603 ms, 15.827 ms, and 17.351 ms. On line 3, only one of the three packets received a reply. The two `*` symbols indicate that the other two ICMP Time-to-Live Exceeded messages were either dropped or suppressed by the router.

The final line (hop 14) shows `172.67.69.208`, which matches the destination IP address for `tryhackme.com` shown in the traceroute header. This means the traceroute completed successfully across 14 total hops. Hops 1 through 13 are intermediate routers, and hop 14 is the destination itself.

Traceroute B

AttackBox Terminal — Traceroute B

```
user@AttackBox$ traceroute tryhackme.com
traceroute to tryhackme.com (104.26.11.229), 30 hops max, 60 byte packets
 1 ec2-79-125-1-9.eu-west-1.compute.amazonaws.com (79.125.1.9) 1.475 ms * ec2
 2 100.65.20.160 (100.65.20.160) 16.575 ms 100.66.8.226 (100.66.8.226) 23.24
 3 100.66.16.50 (100.66.16.50) 2.777 ms 100.66.11.34 (100.66.11.34) 22.288 ms
 4 100.66.6.47 (100.66.6.47) 17.264 ms 100.66.7.161 (100.66.7.161) 39.562 ms
 5 100.66.5.123 (100.66.5.123) 20.099 ms 100.66.7.239 (100.66.7.239) 19.253 m
 6 * 100.66.5.223 (100.66.5.223) 16.172 ms 100.65.15.135 (100.65.15.135) 0.4
 7 100.65.12.135 (100.65.12.135) 0.390 ms 100.65.12.15 (100.65.12.15) 1.045 m
 8 100.100.4.16 (100.100.4.16) 0.482 ms 100.100.20.122 (100.100.20.122) 0.79
 9 100.100.20.86 (100.100.20.86) 0.442 ms 100.100.4.78 (100.100.4.78) 0.347 m
10 100.92.212.20 (100.92.212.20) 11.611 ms 100.92.11.54 (100.92.11.54) 12.67
11 100.92.6.52 (100.92.6.52) 11.427 ms 100.92.6.50 (100.92.6.50) 11.033 ms 1
12 100.92.210.139 (100.92.210.139) 10.026 ms 100.92.6.13 (100.92.6.13) 14.58
13 100.92.79.12 (100.92.79.12) 12.011 ms 100.92.79.68 (100.92.79.68) 11.318 m
14 100.92.9.27 (100.92.9.27) 11.354 ms 100.92.80.31 (100.92.80.31) 13.000 ms
```

```
15 150.222.241.85 (150.222.241.85) 9.660 ms 52.93.135.81 (52.93.135.81) 10.9
16 100.92.228.102 (100.92.228.102) 15.168 ms 100.92.227.41 (100.92.227.41) 1
17 100.92.232.111 (100.92.232.111) 10.589 ms 100.92.231.69 (100.92.231.69) 1
18 100.91.205.140 (100.91.205.140) 11.551 ms 100.91.201.62 (100.91.201.62) 1
19 100.91.205.79 (100.91.205.79) 11.112 ms 100.91.205.83 (100.91.205.83) 11.
20 100.91.211.45 (100.91.211.45) 9.486 ms 100.91.211.79 (100.91.211.79) 13.6
21 100.100.6.81 (100.100.6.81) 11.522 ms 100.100.68.70 (100.100.68.70) 10.18
22 100.100.65.131 (100.100.65.131) 10.371 ms 100.100.92.6 (100.100.92.6) 10.
23 100.100.2.74 (100.100.2.74) 15.317 ms 100.100.66.17 (100.100.66.17) 11.49
24 100.100.16.16 (100.100.16.16) 19.155 ms 100.100.16.28 (100.100.16.28) 19.
25 99.83.89.19 (99.83.89.19) 28.929 ms * 21.790 ms
26 104.26.11.229 (104.26.11.229) 11.070 ms 11.058 ms 11.982 ms
```

In this second run, the packets took a significantly longer route, completing in 26 hops. Hop 26 is the destination `104.26.11.229`, and hops 1 through 25 are intermediate routers. Comparing the two runs shows that neither the number of hops nor the specific routers along the path remained consistent. This is expected behaviour for traffic crossing external networks, especially when CDNs like Cloudflare and Akamai are involved, as they use anycast and load balancing to optimise paths.

There are three key observations from these examples. The number of hops between your system and the target depends on when you run the command. There is no guarantee that packets will follow the same route, even within a short time window, because load balancing, failover mechanisms, and dynamic routing updates can all alter the path. Some routers return public IP addresses that you might examine depending on the scope of your engagement, though these may belong to third parties. Some routers do not return a reply at all, which could be due to rate limiting, firewall rules, or configurations that suppress ICMP responses.

Additional Techniques

Use `mtr 10.113.178.176` (My Traceroute) for a real-time, continuous view that combines traceroute with ping-like statistics, showing packet loss and latency per hop. For bypassing filters, try TCP mode with `traceroute -T 10.113.178.176` or ICMP mode with `traceroute -I 10.113.178.176`. For IPv6, the standalone `traceroute6` command ensures compatibility with dual-stack networks.

Q 6-In Traceroute A, what is the IP address of the last router/hop before reaching tryhackme.com?

Answer : 100.92.9.83

Q 7- In Traceroute B, what is the IP address of the last router/hop before reaching tryhackme.com?

Answer : 99.83.89.19

Q 8-In Traceroute B, how many routers are between the two systems?

Answer : 25

Task 5- Telnet

The **TELNET** (Teletype Network) protocol was developed in 1969 to communicate with a remote system via a command-line interface. The `telnet` command uses this protocol for remote administration, with a default port of 23. From a security perspective, telnet sends all data in cleartext, including usernames and passwords. This makes it trivial for anyone with access to the communication channel to intercept login credentials. The secure alternative is **SSH** (Secure Shell), which encrypts all traffic and is the standard for remote CLI access today.

Despite its security shortcomings, the telnet client has a useful property for reconnaissance. Because it operates over TCP, you can use it to connect to any TCP port and observe the server's response. This technique is known as **banner grabbing**. You connect to a service and read the initial response, called the "banner", that the server sends back. Banners frequently reveal the software name and version running on that port.

If `telnet` is not installed on your system, you can install it on Debian and Ubuntu with `apt install telnet`. However, `netcat` (`nc`) and `curl` are generally preferred alternatives that provide similar functionality with more flexibility.

The example below demonstrates banner grabbing against a web server on port 80. You connect with `telnet 10.113.178.176 80`, then issue a minimal HTTP request. The command `GET / HTTP/1.1` followed by `host: example` and two presses of the Enter key is sufficient to elicit a response.

Note: The terminal output below uses an example web server running nginx for

illustration purposes. The VM attached to this task runs a different web server. When you perform the exercise, your output will differ from this example.

Terminal

```
pentester@TryHackMe$ telnet 10.113.178.176 80
Trying 10.113.178.176...
Connected to 10.113.178.176.
Escape character is '^]'.
GET / HTTP/1.1
host: telnet
```

```
HTTP/1.1 200 OK
Server: nginx/1.6.2
Date: Tue, 17 Aug 2021 11:13:25 GMT
Content-Type: text/html
Content-Length: 867
Last-Modified: Tue, 17 Aug 2021 11:12:16 GMT
Connection: keep-alive
ETag: "611b9990-363"
Accept-Ranges: bytes
...
```

The field of interest here is `Server: nginx/1.6.2`, which reveals the type and version of the web server software. This is exactly the kind of information that active reconnaissance aims to uncover. Software versions can be cross-referenced against known vulnerabilities in databases such as CVE and Exploit-DB.

This same banner-grabbing technique works against any TCP-based service. If you connect to a mail server, you would use SMTP or POP3 commands instead of HTTP. If you connect to an FTP server on port 21, the server typically sends its banner immediately upon connection without requiring any commands at all. The underlying principle is always the same. Connect to the port, read what the server sends back, and optionally issue protocol-specific commands to extract further information.

In modern environments, many services enforce encryption. For example, SMTPS runs on port 465, and HTTPS runs on port 443. Telnet cannot handle encrypted connections. For HTTPS, use `curl --head https://10.113.178.176` or

`openssl s_client -connect 10.113.178.176:443` . For other TLS-wrapped services, `openssl s_client` or `ncat --ssl` are the appropriate tools.

Q 9- Start the attached VM from Task 3 if it is not already started. On the AttackBox, open the terminal and use the telnet client to connect to the VM on port 80. What is the name of the running server?

Answer : Apache

Q 10-What is the version of the running server (on port 80 of the VM)?

Answer : 2.4.61

Task 6- Netcat

Netcat (or simply `nc`) is a versatile networking utility that supports both TCP and UDP protocols. It can function as a client that connects to a listening port, or as a server that listens on a port of your choice. This dual capability makes it useful for banner grabbing, port probing, simple file transfers, and basic client-server communication. Modern versions such as `ncat` from the Nmap project also support IPv6 and SSL encryption, making it more versatile than legacy tools like telnet.

Note: This task uses its own VM. If you still have a VM running from a previous task, you may need to terminate it before starting the one associated with this task. The question below will instruct you to start the VM and the AttackBox.

Banner Grabbing with Netcat

The banner-grabbing technique described in the previous task works identically with `nc` . The syntax is `nc MACHINE_IP PORT` . You connect to the target port, then issue protocol-appropriate commands to read the server's response. Note that you might need to press **Shift+Enter** after the GET line.

Terminal

```
pentester@TryHackMe$ nc MACHINE_IP 80
GET / HTTP/1.1
host: netcat
```

```
HTTP/1.1 200 OK
Server: nginx/1.6.2
Date: Tue, 17 Aug 2021 11:39:49 GMT
Content-Type: text/html
Content-Length: 867
Last-Modified: Tue, 17 Aug 2021 11:12:16 GMT
Connection: keep-alive
ETag: "611b9990-363"
Accept-Ranges: bytes
...
```

In this example, `nc MACHINE_IP 80` opens a TCP connection to port 80. The HTTP request `GET / HTTP/1.1` followed by `host: netcat` produces a response that includes `Server: nginx/1.6.2`, revealing the web server software and version.

This same approach applies to any TCP-based service. Connecting to an FTP server on port 21 with `nc MACHINE_IP 21` will typically produce an immediate banner showing the FTP server software and version without requiring any commands. Connecting to an SMTP server on port 25 produces a banner identifying the mail server. The principle is consistent across protocols. Connect, read the banner, and optionally issue protocol-specific commands.

Listening with Netcat

Netcat can also act as a server, listening on a specified port. This is useful for testing connectivity, transferring simple data, or setting up basic communication channels during an engagement.

On the server system, run `nc -vnlp 1234` to start listening on port 1234. On the client system, run `nc MACHINE_IP 1234` to connect. Once the connection is established, any text typed on one side is transmitted to the other. As you may recall from the Linux Fundamentals module, the exact order of the flags does not matter as long as the port number is preceded directly by `-p`.

Option	Meaning
`-l`	Listen mode
`-p`	Specify the port number
`-n`	Numeric only; no DNS resolution of hostnames
`-v`	Verbose output, useful for debugging
`-vv`	Very verbose output
	Keep listening after the client disconnects

The `-p` flag must appear directly before the port number. The `-n` flag avoids DNS lookups and associated warnings. Port numbers below 1024 require root privileges to listen on. For IPv6 listening, add the `-6` flag with `nc -6 -lp 1234`. If you need encryption for sensitive data transfer, use `ncat --ssl` or pair `nc` with a tool like `stunnel`.

Q 11- Start the VM and open the AttackBox. Once the AttackBox loads, use Netcat to connect to the VM port 21. What is the version of the running server?

Answer : 0.17

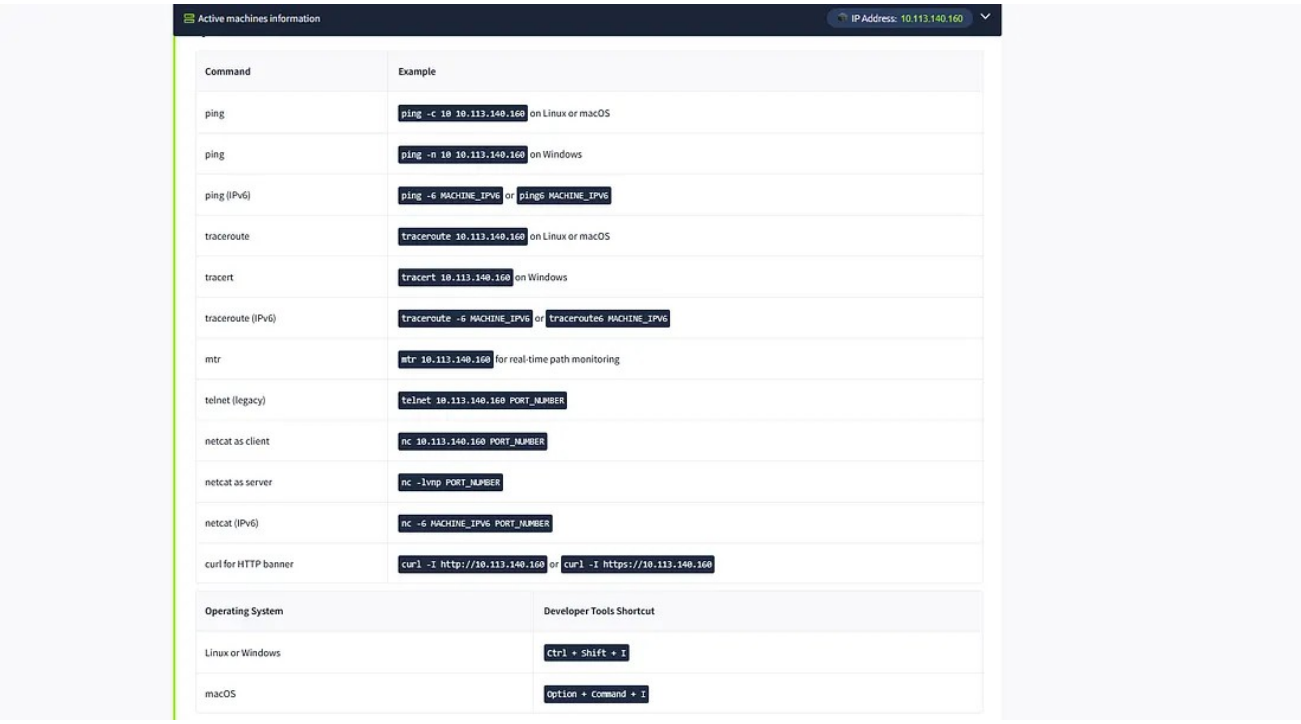
Task 7- Putting It All Together

This room covered five core tools for active reconnaissance. The web browser with Developer Tools reveals server technologies, headers, JavaScript sources,

and certificate details. `ping` confirms whether a target is reachable and provides TTL-based clues about its operating system. `traceroute` maps the network path between you and the target, revealing intermediate routers and potential filtering points. `telnet` and `netcat` connect to individual ports to grab banners and identify running services along with their versions.

These tools are simple individually, but combining them gives you a structured picture of a target before moving on to more advanced scanners. You might use `ping` to confirm a host is alive, `traceroute` to understand the network path, and then `nc` to probe specific ports and identify services. For HTTP-based services, prefer `curl -I 10.113.140.160` or `nc 10.113.140.160 PORT` over telnet for banner grabbing, as they provide more secure and flexible options.

Quick Reference



happy hacking

Nuh hozar - Medium

Read writing from Nuh hozar on Medium. Whatever is in the name of goodness and beauty, those who know us know, those...

nuhhozar.medium.com

nuh hozar block sayfası

https://nuhhozar.blogspot.com

nuhhozar.blogspot.com

https://github.com/nuhhozar

Active Reconnaissance +



Follow

Written by Nuh hozar

5 followers · 9 following

Whatever is in the name of goodness and beauty, those who know us know, those who don't know know as well as themselves.‼